

Assignment

1. What problem was EF solving compared to [ADO.NET](#)?

First of all [ADO.Net](#) stands for Activex Data Object which is a Data Access technology developed by Microsoft. Here one can connect with the database with the help of it , then Insert , delete, update and retrieve the data accordingly.

Problems By [ADO.Net](#) and Solved by EF : Entity Framework is useful to remove major pitfalls of [ADO.Net](#) which is very costly to deal with.

1) Remove Manual Mapping.

For e.g :- [emp.Name](#) = Reader["Name"];

Rather we can do

:- [emp](#) = dbContext.Employees.ToList();

2) Automatic Relationship Handled.

Here we do not have to perform manual joins between two tables , using Navigation property we can automate the process of mapping.

3) Remove SQL dependencies from application code.

Here we are not writing SQL queries manually rather we write LINQ queries which later convert into SQL queries using [ADO.Net](#) internally.

4) Provide Change Tracking.

Here proper Change tracking property provided by EF core which mainly remember the different State of the data and then maps it with object

5) Data Base independence.

It is not strongly depends on one particular Database syntax like [ADO.Net](#) Depending on SQL , with the help of simple changes we can switch into multiple databases.

6) Reduce Boiler plate code.

It Reduces major boilerplate code which we wrote in case of [ADO.Net](#) like connections to db and close connection to db , then reader and adapter depending on case.

2. Explain EDM architecture (Conceptual, Mapping, Storage).

Entity Data Model is a part of Entity Framework Architecture it holds three major components

- 1) Conceptual Model :- The conceptual model contains data classes and their Relationships . It's not depends on database Entity, both are independent
- 2) Storage Model :- This is a model having Data Tables , Database Designs, Store Procedure , Table Relationships and Keys.
- 3) Mapping :- It includes information about how the conceptual model is mapped to the storage mode.
- 4) LINQ to entities :- it is a query language used to write queries against the object model It returns an entity which is defined in conceptual Mode. We can apply various LINQ operations like SELECT , WHERE , GROUP BY etc ... to retrieve or filter the data.
- 5) Entities to SQL :- Here Entity SQL is another query language , which is little more difficult Than L2E . which is generally used for EF .net 6
- 6) Object Services :- It works as an entry point for Data Access from the db and in return it, sql entities came from data bases are materialized into the object entity over here.
- 7) Entity Client Data Provider:- The main responsibility of this layer is to convert LINQ to Entity sql queries or Entity sql queries to sql queries which is understood by the underlying database.
- 8) [ADO.Net](#) Data Provider :- This layer communicates with the databases using standard ADO.NET

3. Difference between EF Core and EF6.

Aspect	Entity Framework 6	Entity Framework Core
Platform Support	It will support only Windows Operating systems.	Here it is crossplatform ORM ,Support multiple operating systems
Architecture	Older & monolithic Architeccture	Lightweight , modular , redesigned from scratch
Performance	Slower , take more time to run	Faster and optimized SQL generation
Development Approach	Designed by traditional enterprise apps	Built for modern , cloud base scalable apps
Configuration	Use EDMX designer heavily	Code first approach
Use Cases	Maintaining Legacy applications	It's Mostly use in current development

4. What happens internally when SaveChanges() is called?

When a savechanges() call then this call tracked what changes were made . it create SQL Queries and perform SQL operation according to the given changes
Like if we added something than internally Insert into query call take place
If modification perform then update takes place , for delete operation delete take place
Thats how SaveChange() work internally

- It's also check whether each fields are presents
- it's check whether valid data added or not
- it's also check the referential and integrity constraint

5. What is Change Tracking?

In a simple word it is a mechanism to track the changes within an entity loaded inside dbContext, and determine what modifications done to them and determine database operations like insert , delete or update need to perform when SaveChanges() function call occurs.

For an example :-

Code

```
Var emp = dbConext.Employee.first();
```

Original values snapshot (from DB)

Current Values (Live Object)

```
emp.salary = 70000
```

EF compare

Original : salary = 50000

Current : salary = 70000

It change the status to

EntityState.Modified

There are various state of Entity which is maintained by change tracker

State Added :- New entity

Modified :- Updated entity

Deleted :- Remove entity

Unchanged :- No change

Detached :- No tracked

This Change status store inside :- context.ChangeTracker.

If we use AsNotTrack() method then it will stop track changes and treat data as readonly changes which we perform on the data are not tracked ,monitored by tracker even if we use SaveChange() function.

6. Difference between DbContext and DbSet?

Aspect	DbContext	DbSet<TEntity>
Definition	It represents sessions with the database and manages all operations.	Represent a specific table in db.
Role	Act as the unit of work & change tracker coordinator.	Acts like a repository for a single entity type
Mapping	Knows the entire model and relationships.	Known only one entity mapping.
SaveChanges()	Exist here - commits all tracked changes.	Can not save independently.
Responsibility	Manage Connections, tracking, transections and saving changes	Perform CRUD Operations for that entity only.